

Title

Ubuntu Core Support Process

Type	Author(s)	Status	Created
Process ▾	Ernest Lotter	Pending Review ▾	Jan 27, 2026
Reviewer(s)	Status	Date	
Alfonso Sanchez-Be...	Approved ▾	📅 Date	
Samuele Pedroni	Pending Review ▾	📅 Date	
Mohit Chachada	Approved ▾	Apr 24, 2026	
Nick Dyer	Pending Review ▾	Feb 3, 2026	

Abstract

This document describes the support lifecycle for Ubuntu Core. This is aligned with the Ubuntu Classic LTS release lifecycle. This specification defines such a lifecycle and aligns also snapd, as a core component of Ubuntu Core, with it to ensure consistent maintenance, security, and support guarantees.

Rationale

Historically, snapd has been updated independently of Ubuntu Core releases. As a result, significantly newer versions of snapd could be deployed on comparatively old Ubuntu Core releases. While this flexibility has offered convenience, it has also produced an expanding matrix of version combinations that cannot be comprehensively tested or supported. As Ubuntu Core releases age and snapd evolves, sustaining compatibility across such an unconstrained matrix becomes increasingly difficult, diverting engineering effort from supported releases and undermining predictable security coverage, stability, and platform evolution.

Defining an explicit support lifecycle constrains the supported Ubuntu Core and snapd combinations by tying snapd releases to specific Ubuntu Core versions through dedicated

tracks and maintenance branches.

Existing Ubuntu Core support documentation describes the Ubuntu Core support lifecycle at a high level, including support phases and durations. This specification adds further detail to that lifecycle model and defines the operational processes used to deliver and maintain Ubuntu Core releases across their supported lifetimes, as well as the mechanisms used to transition existing releases to the lifecycle model defined here.

Specification

This document is a normative process specification defining lifecycle alignment and maintenance rules for Ubuntu Core releases and their components, including snapd when used as part of Ubuntu Core. Where it differs from existing practice, it takes precedence.

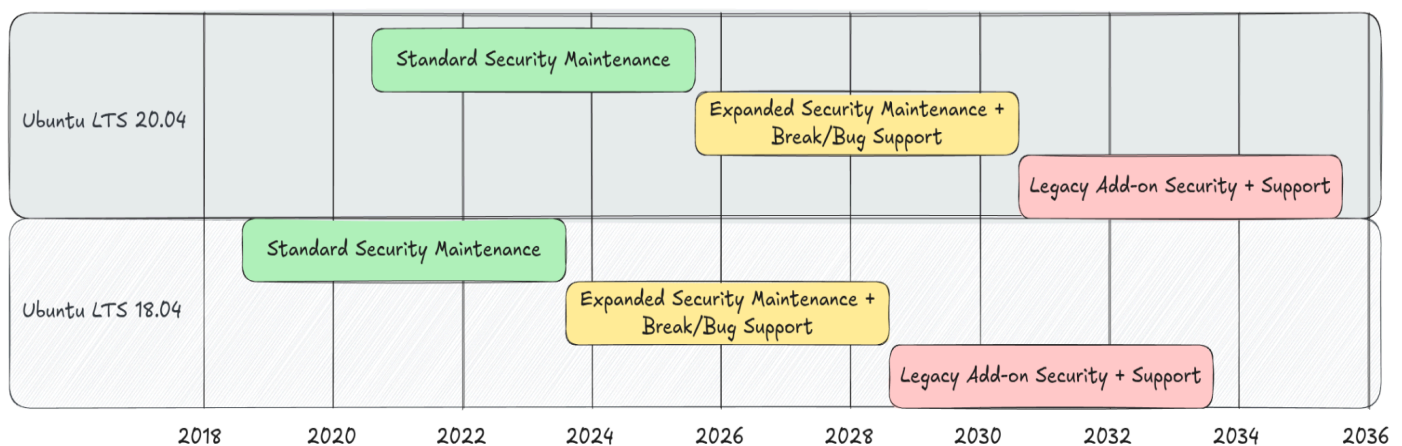
Alignment with Ubuntu Classic Support Lifecycle

The support lifecycle of Ubuntu Core — comprising the core base, gadget, ubuntu-core-initramfs, kernel, snapd, and system snaps — aligns with Ubuntu Classic LTS releases in terms of overall lifespan, start of Standard Security Maintenance as well as having similar Legacy Add-on phase.

The key difference is that Ubuntu Core does not have a separate Expanded Security Maintenance (ESM) phase. Instead, it provides 10 years of Standard Security Maintenance (SSM), covering the duration of the Ubuntu Classic SSM and ESM phases.

Ubuntu Classic Support Lifecycle Phases

- Standard security maintenance (SSM): 5 years
- Expanded security maintenance (ESM) and break/bug support: +5 years
- Legacy add-on security maintenance and support: + 5 years



Ubuntu Core Support Lifecycle Phases

- Standard security maintenance (SSM): 10 years
- Legacy add-on security maintenance and support: + 5 years

Initial Release Preparation

An Ubuntu Core release <UC-version> must be ready for the initial release at the same time as the corresponding Ubuntu Classic release.

Standard Security Maintenance

Kernel, gadget, and system snaps use <UC-version>/<risk> tracks throughout their supported lifetimes, while core base snaps are provided as distinct snaps named core<UC-version> rather than using tracks.

SSM is divided into two five-year sub-phases:

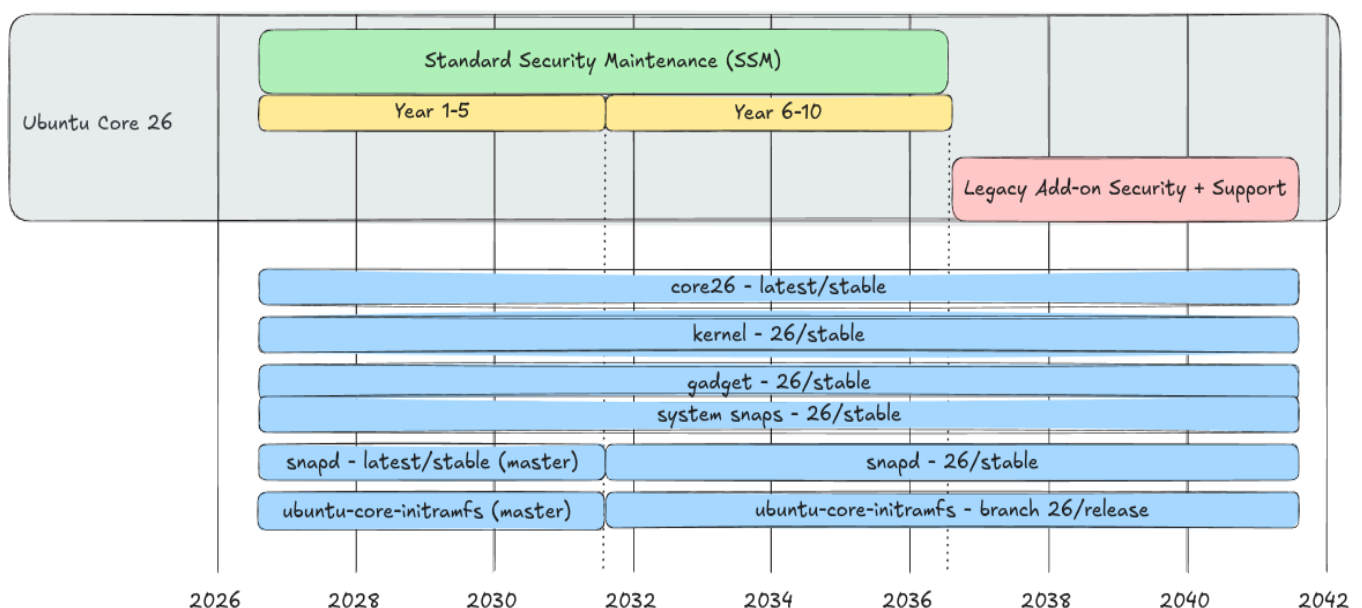
- Year 1 to 5: During the first half of the SSM phase, snapd and ubuntu-core-initramfs builds follow the snapd master branch.
- Year 6 to 10: The preferred mechanism for providing security maintenance and bug fix support for Ubuntu Core components during the second half of the SSM phase is the use of LTS tracks and corresponding maintenance branches.

From this phase snapd and the ubuntu-core-initramfs must adopt a similar model. Snap specific requirements are defined in the [snapd section](#).

Legacy Add-on Security Maintenance and Support

This phase continues to use LTS tracks and corresponding maintenance branches similar to the **year 6 to 10 phase**, but may offer different levels of support.

Ubuntu Core Support Lifecycle:



Scope

Deliverables

Multiple teams are involved in producing and maintaining the Ubuntu Core components and artifacts that together form the set of deliverables that must be maintained over the lifetime of a release.

The Ubuntu Core deliverables covered by this specification are:

- core base snap
- gadget snap
- ubuntu-core-initramfs debian package
- kernel snap (built using ubuntu-core-initramfs)
- snapd snap
- system snaps
- testing platform images (and associated [reference models](#))

Teams and responsibilities:

TEAMS	RESPONSIBILITIES
Ubuntu Core Team	Delivering core base, gadget, ubuntu-core-initramfs and system snaps, except network-manager
Server Team	Delivering network-manager
Snapd Team	Delivering snapd snap
Snapd Team	Testing snapd and Ubuntu Core and maintaining snapd CI on release/lts/<UC-version> branches in the GitHub private snapd fork to ensure continued release readiness of snapd <UC-version>/<risk> tracks for the remainder of the Ubuntu Core support lifecycle.
Kernel Team	Delivering kernel snaps, which depend on ubuntu-core-initramfs
Ubuntu Core Team	Delivering testing platform images and associated reference models
Certification Team	Testing of snapd, gadget, base and kernel snaps and testing platform images. Also the system snaps if they have been part of a customer image.

The Ubuntu Core team is responsible for all coordinating activities across these teams.

Ubuntu Core System Snaps

The Ubuntu Core team is responsible for the following [system snaps](#): bluez, modem-manager, udisks2, upower, and console-conf. The Server team is responsible for maintaining and delivering network-manager. Other system snaps are not managed by the Ubuntu Core team and are therefore outside the scope of this document.

Ubuntu Core Gadget Snaps

The Ubuntu Core team is only responsible for a subset of gadgets as per the [Maintained gadget snaps](#) specification. Gadget snaps that are not managed by the Ubuntu Core team are outside the scope of this document.

Testing Platforms

It is required to produce official test images for people trying out Ubuntu Core. There are currently 4 maintained [testing platforms](#), spanning the amd64, arm64, and armhf (up to UC22) architectures.

Certified Platforms

Support of certified platforms is not considered part of the Ubuntu Core support lifecycle defined by this specification, because support for customer platforms can be requested at any point in time, and is not in step with any lifecycle phases. Requests are supported by the HW enablement team, including creating images to be used for certification testing.

Once a platform has been enabled, it becomes part of the scope of release certification testing for snapd and other Ubuntu Core components for the duration of the platform's lifetime. After this period, the platform's continued relevance should be reassessed.

Commitments

Commitments for the different lifecycle phases:

PHASE		COMMITMENTS
Initial Release Preparation		• Roadmap commitments including new Ubuntu Core and snapd features • All Ubuntu Core deliverables
Standard Security Maintenance	Year 1 to 5	• Ubuntu Core features that are compatible with the LTS version, including updates and enhancements to those features. • Snapd features and feature enhancements • Interfaces and interface enhancements • Best effort bug fixes • Security fixes as required by SSDLC policy

	Year 6 to 10	• Best effort bug fixes • Security fixes as required by SSDLC policy • Ubuntu Core deliverables: ◦ ubuntu-core-initramfs LTS branch ◦ snapd LTS track ◦ test platform images pointing to above LTS tracks
Legacy Add-on Security + Support 10–15		• Security fixes as required by SSDLC policy • No bug fixes

If a product requires newer Ubuntu Core or snapd features after **year 1 to 5**, the way to achieve this is via remodeling to a newer Ubuntu Core version.

Initial Release Scheduling

The purpose of this schedule is to ensure the timely delivery of all Ubuntu Core components while accounting for upstream and downstream dependencies. It defines the expected sequencing and target timing of release activities and serves as the baseline for planning and coordination.

This schedule must be used as the starting point for each Ubuntu Core release and may be refined as necessary to maintain alignment with the corresponding Ubuntu Classic release and to reflect improved understanding of dependencies and constraints.

Reference Ubuntu Core Release Schedule:

DATE	UBUNTU CLASSIC EVENT	UBUNTU CORE EVENT	SNAPD EVENT
Mid Oct	Toolchain Uploaded		
End Nov	Classic Snapshot 1	Build coreXX and gadget on edge. Build ubuntu-core-initramfs. Request kernel snap builds and get them published to edge.	Start adapting automated tests for XX.04
Mid Dec	Classic Snapshot 2	Deliver images that can run smoke tests.	
Early Jan			Update dependencies
Mid Jan			
End Jan	Classic Snapshot 3	Update coreXX and gadget on edge. Update ubuntu-core-initramfs.	

		Request kernel snap updates on edge.	
Mid Feb	Feature Freeze		
End Feb	Classic Snapshot 4	Start release of coreXX, gadget, and system snaps. Deliver ubuntu-core-initramfs, aligned with snapd, to the kernel team. Request release of kernel snaps. All this for the edge channel.	Start snapd release
Mid March	UI freeze		
Mid March + few days	Kernel Feature Freeze		
End March	Beta Freeze	Deliver kernel, coreXX, gadget and system snaps to the stable channels. Create testing platform images and send them to cert testing.	Deliver snapd in stable
Mid April	Final Freeze	Solve any bugs found during testing and re-release affected snaps.	
End April	Release		

Updates

All deliverables must be produced for the initial delivery. Thereafter updates are driven by changes in the component or its upstream or downstream dependencies.

Update triggers for the different deliverables during the different phases:

DELIVERABLE	YEAR 1 TO 5 TRIGGERS (1-5)	YEAR 6 TO 10 TRIGGERS (6-10)	LEGACY TRIGGERS (11-15)
Testing platform images	Changes to any of the components. Currently, testing platform images are manually rebuilt on request only	Changes to any of the components. Currently, testing platform images are manually rebuilt on request only	Changes to any of the components. .Currently, testing platform images are manually rebuilt on request only
core base and	Security and bug fixes	Security and bug fixes	Security and bug fixes to

system snaps	to their classic counterparts or snap specific fixes	to their classic counterparts or snap specific fixes	their classic counterparts or snap specific security fixes
gadget snap	Security and bug fixes	Security and bug fixes	Security fixes
ubuntu-core-initramfs	snap-bootstrap updates from snapd master, which in turn also depends on secboot	Security and bug fixes	Security fixes
kernel	Security and bug fixes to Ubuntu Classic counterpart or ubuntu-core-initramfs changes	Security and bug fixes to Ubuntu Classic counterpart or ubuntu-core-initramfs changes	Security and bug fixes to Ubuntu Classic counterpart or ubuntu-core-initramfs changes
Snapd	Snapd features and feature enhancements, security and bug fixes and dependency updates such as AppArmor or Secboot.	Security and bug fixes in snapd or dependencies such as AppArmor or Secboot.	Security fixes in snapd or dependencies such as AppArmor or Secboot.

Snapd

Tracks and Git Branching

The snapd release [channels](#) consist of <track>/<risk>.

When an Ubuntu Core version reaches **year 6**, snapd must transition from using the latest/stable track to <UC-version>/<risk> tracks. This is the same convention that is already used for system snaps in general e.g. Bluez.

In order to transition devices running Ubuntu Core to snapd on the corresponding track, a mechanism is required to force snapd onto the track matching the Ubuntu Core version. This mechanism must ensure that snapd switches tracks before any further updates to other snaps are applied. The design and implementation of this mechanism are defined in a separate specification: [UC042 - Snapd ESM tracks for Ubuntu Core](#).

As part of entering the year 6 to 10 phase, a dedicated snapd LTS Git branch must be created for that Ubuntu Core version. It makes sense to unify the snapd security release processes for Ubuntu Classic and Ubuntu core to the extent where the same LTS Git branch can be used for building snaps and debs. This is taken into account in the LTS branch naming.

LTS Git branches:

- Must be named release/lts/<UC-version> e.g. release/lts/22

- Must be created in the private snapd fork from the most recent public release/<X.YY> branch available at the time of transition.
- If more than one UC-version needs to use the same branch, it is still required to create separate release branches for each <UC-version> from the same public release branch. This simplifies automation by preventing the need for special case logic.
- Snapd versions are marked using GIT tags.
- Must exist in both the [GitHub private snapd fork](#) and the [Launchpad private snapd fork](#).

The GitHub private snapd fork must be used for preparing fixes and running automated tests, while the Launchpad private snapd fork must be used for building snapd artifacts.

CI Maintenance

All active snapd LTS Git branches on the [GitHub private snapd fork](#) require regular CI build and spread test maintenance.

When a new LTS Git branch is created in the private snapd fork:

- Snapd spread tests targeting <UC-version> should be removed from the public repository.
- A smoke test for <UC-version> should remain in the public repository until backward compatibility is intentionally broken or discontinued (for example due to non-compatible features or Go version changes).

All LTS Git branches in the private snapd fork must receive routine spread test maintenance to ensure they remain ready for future releases. This includes backporting relevant test fixes from the master branch.

Some test dependencies should be pinned to prevent tests from breaking frequently as the environment ages. Details will be defined in a separate specification:

<https://warthogs.atlassian.net/browse/SNAPDENG-36043> <add spec here when available>.

Updates

Non-security snapd releases currently follow a two-monthly release cadence that is compatible with Ubuntu Classic release timings. Low priority security fixes are accumulated and released towards the end of a cycle. High priority security and bug fixes must happen as soon as possible, preferably shortly after a major release, to avoid delaying the next major release.

Security vulnerabilities in snapd and any of its source or packaged dependencies must be covered by snapd security maintenance. These vulnerabilities should be identified through SSDLC vulnerability scanning practices.

Releases containing security fixes must be prepared in accordance with the [Snapd Security](#)

[Release Process](#). Bug fixes should be made upstream first, where applicable. If an upstream fix is not applicable, the fix must be prepared against the most recent snapd LTS Git branch in which the issue exists. In all cases, bug fixes must be backported to all active snapd LTS branches. Where a fix cannot be reasonably applied to a given LTS branch, this must be documented as part of the release process.

Transitioning to Tracks

Ubuntu Core 18 and Ubuntu Core 20 have already reached **year 6** and are behind in terms of using tracks and corresponding branches. Ubuntu Core 16 with its core OS snap is special, and should continue to follow the [Snapd Security Release Process](#) to update the core OS snap.

The following transitions needs to happen as soon as possible:

UBUNTU CORE VERSION	ACTION	BRANCH FROM	TARGET DATES
Ubuntu Core 18 (reached year 6 on 2 May 2023)	Create 18/<risk> track and release/lts/18 branch	2.7X (latest)	Nov 2026
Ubuntu Core 20 (reached year 6 on 2 May 2025)	Create 20/<risk> track and release/lts/20 branch	2.7X (latest)	Jan 2027

Snapd for Ubuntu Core/Classic 16 already has a branch [core-snap-security-release/2.61](#).

Automation

The Ubuntu Core support lifecycle, which includes the transition to snapd LTS tracks, adds additional workload and time constraints to the Ubuntu Core and Snapd Teams.

Time-consuming activities:

TEAM	TIME CONSUMING ACTIVITIES
Snapd / Ubuntu Core	Adapting CI builds and tests for Ubuntu Classic and Ubuntu Core releases
Snapd / Ubuntu Core	CI build and test maintenance including the snapd private repository that requires backporting of test applicable bug fixes
Snapd / Ubuntu Core	Re-running flaky tests
Ubuntu Core	Backporting security and bug fixes to older core versions and gadgets, ubuntu-core-initramfs and system snaps
Ubuntu Core	Manual verification of testing platform images on real hardware before requesting certification

	testing
Snapd	Backporting security and bug fixes to older snapd LTS Git branches
Snapd / Ubuntu Core	Generating high quality release notes
Snapd / Ubuntu Core	Many manual release steps

These activities motivate the need for further automation, which should be addressed in follow-up specifications.

Further Information

- [Ubuntu Classic release cycle](#)
- [Ubuntu Kernel release cycle](#)
- [Ubuntu Docs - support duration](#)
- [Snapd Security Release Process](#)
- [Ubuntu Pro Description](#)

Spec History and Changelog

Please be thorough when recording changes and progress with the spec itself and the work resulting from it. Record every meeting, attendees and conclusions from the meeting.

Author(s)	Status	Date	Comment
Ernest Lotter	Braindump ▾	Jan 20, 2026	Brain dump
Ernest Lotter	Drafting ▾	Jan 27, 2026	Draft
Ernest Lotter	Drafting ▾	Mar 5, 2026	Many improvements based on review comments and discussions. Ready for another review iteration.
Ernest Lotter	Drafting ▾	May 5, 2026	Updated SSM phase 1 & 2 to Year 1-5 and Year 6-10 following discussion with Gabriel.

